GOOGLE CLOUD PROFESSIONAL CLOUD ARCHITECT

Section 2.1 — Planning and Configuring Compute Resources

GCP Associate Cloud Engineer

Study Notes Study Notes April 2026

Exam Relevance

This topic is part of **Section 2: Planning and configuring a cloud solution (~17.5 % of the exam)**. You must be able to select the appropriate compute option for a given workload and understand when to use Spot VMs and custom machine types.

1. Google Cloud Compute Options Overview

Google Cloud offers a spectrum of compute services, from fully managed infrastructure (IaaS) to fully managed serverless (FaaS):

Decision Matrix

Criteria	Compute Engine	GKE	Cloud Run	Cloud Functions
Workload type	Any (VMs)	Containerized microservices	Containerized stateless apps	Event-driven functions
Management level	You manage OS, runtime, app	You manage containers, K8s config	Google manages infra	Google manages everything
Scaling	Manual/Autoscaler	Pod/Node autoscaling	Automatic (0 to N)	Automatic (0 to N)
Startup time	Minutes	Seconds (pods)	Seconds	Seconds
Pricing	Per second (min 1 min)	Per cluster + node VMs	Per request + CPU/memory time	Per invocation + compute time
State	Stateful	Stateful/Stateless	Stateless (preferred)	Stateless
Max request timeout	Unlimited	Unlimited	60 min (gen2)	9 min (gen1) / 60 min (gen2)
Use cases	Legacy apps, GPU, custom OS	Microservices, hybrid, multi-cloud	Web APIs, async processing	Webhooks, triggers, lightweight processing

2. Compute Engine (Virtual Machines)

What Is Compute Engine?

- **IaaS** — Google's virtual machine service
- Full control over the operating system, software stack, and configuration
- Runs on Google's global infrastructure

Machine Type Families

Family	Prefix	Best For
General-purpose	E2, N2, N2D, N1, T2D, T2A, C3, C3D	Balanced workloads (web servers, databases, dev environments)
Compute-optimized	C2, C2D, H3	CPU-intensive (gaming, HPC, batch processing, media transcoding)
Memory-optimized	M1, M2, M3	Large in-memory workloads (SAP HANA, in-memory databases)
Accelerator-optimized	A2, A3, G2	ML training/inference, GPU workloads
Storage-optimized	Z3	High IOPS, large local SSD workloads

Machine Type Naming Convention

```
FAMILY-STANDARD-CPUS
Example: e2-standard-4
         |  |         |
         |  |         └── 4 vCPUs
         |  └── Standard ratio (CPU:Memory)
         └── E2 family
```

Ratio Options: - `standard` — 4 GB per vCPU - `highmem` — 8 GB per vCPU - `highcpu` — 1 GB per vCPU - `micro/small/medium` — Shared-core types

Custom Machine Types

- Create VMs with **non-standard CPU and memory configurations**
- Available for N1, N2, N2D, and E2 families
- Specify exact vCPU count and memory (in 256 MB increments)
- **Use case**: When predefined types waste resources (e.g., need 6 vCPUs with 20 GB RAM)

```
# Create a custom machine type
gcloud compute instances create my-vm \
  --custom-cpu=6 \
  --custom-memory=20GB \
  --zone=us-central1-a

# Or using the machine type string
gcloud compute instances create my-vm \
  --machine-type=custom-6-20480 \
  --zone=us-central1-a
```

Pricing: Custom machine types cost slightly more per resource unit than predefined types of the same family.

Sole-Tenant Nodes

- Physical server dedicated exclusively to your VMs
- Use cases: Compliance, licensing (BYOL), performance isolation
- Specify a node template and node group

When to Choose Compute Engine

- You need full control over the OS and runtime environment
- Running legacy applications that cannot be containerized
- Workloads requiring GPUs or TPUs
- Stateful applications needing persistent local storage
- Specific OS or kernel requirements
- Windows Server workloads
- Lift-and-shift migrations from on-premises

---

3. Spot VMs (formerly Preemptible VMs)

What Are Spot VMs?

- **Significantly cheaper** VMs (60-91% discount compared to on-demand)
- Google can **reclaim them at any time** (with a 30-second warning)
- Maximum lifetime of **24 hours** (Preemptible) or **no maximum** (Spot)
- Not covered by SLA

Spot VMs vs. Preemptible VMs

Feature	Spot VMs	Preemptible VMs (Legacy)
Max lifetime	No limit (but can be reclaimed)	24 hours
Pricing model	Dynamic (market-based)	Fixed discount
Availability	Same	Same
Reclaim notice	30 seconds	30 seconds

When to Use Spot VMs

- **Batch processing** — Jobs that can be checkpointed and restarted
- **CI/CD pipelines** — Build and test jobs
- **Data processing** — Hadoop, Spark, Dataproc workloads
- **Rendering** — Video/image rendering farms
- **Machine learning training** — Distributed training with checkpointing
- **Dev/test environments** — Non-critical development workloads

When NOT to Use Spot VMs

- Production web servers serving live traffic
- Databases requiring high availability
- Applications that cannot tolerate interruptions
- Workloads with strict SLA requirements

gcloud Commands

```
# Create a Spot VM
gcloud compute instances create my-spot-vm \
  --provisioning-model=SPOT \
  --instance-termination-action=STOP \
  --zone=us-central1-a \
  --machine-type=e2-standard-4

# Create a Preemptible VM (legacy)
gcloud compute instances create my-preemptible-vm \
  --preemptible \
  --zone=us-central1-a

# Termination actions:
# STOP — VM is stopped (can be restarted if capacity available)
# DELETE — VM is deleted
```

Handling Preemption

- Use **shutdown scripts** to gracefully handle termination
- Implement **checkpointing** for long-running jobs
- Use **managed instance groups** with Spot VMs for automatic replacement
- Monitor the **metadata server** for preemption notices

```
# Check preemption status from within the VM
curl -H "Metadata-Flavor: Google" \
  http://metadata.google.internal/computeMetadata/v1/instance/preempted
```

---

4. Google Kubernetes Engine (GKE)

What Is GKE?

- **Managed Kubernetes** service on Google Cloud
- Runs containerized applications using Kubernetes orchestration
- Google manages the control plane; you manage (or Google manages) the worker nodes

GKE Modes

Feature	Standard	Autopilot
Node management	You manage	Google manages
Pricing	Pay for nodes (VMs)	Pay per pod (CPU, memory, storage)
Node configuration	Full control	Google-optimized
Security hardening	Manual	Automatic
Scaling	Manual + autoscaler	Fully automatic
Best for	Custom requirements, full control	Most workloads, reduced ops overhead

When to Choose GKE

- Running **microservices architectures**
- Need **container orchestration** (auto-healing, rolling updates, service discovery)
- **Hybrid or multi-cloud** deployments (GKE Enterprise / Anthos)

- Teams already using Kubernetes
- Need **GPU workloads** in containers
- Complex applications with multiple services that need to communicate
- Applications requiring **stateful sets** (databases in containers)

Key Concepts for the Exam

- **Pods** — Smallest deployable unit (one or more containers)
- **Nodes** — VMs that run pods
- **Node Pools** — Groups of nodes with the same configuration
- **Clusters** — Collection of nodes managed by a control plane
- **Services** — Stable network endpoint for a set of pods
- **Deployments** — Declarative management of pod replicas

---

5. Cloud Run

What Is Cloud Run?

- **Fully managed serverless** platform for running containers
- Scales automatically (including to zero)
- Pay only for resources used during request handling
- Based on the Knative open-source project

Key Features

- Deploy any container image (any language, any framework)
- Automatic HTTPS endpoints
- Scales from 0 to thousands of instances
- Maximum request timeout: 60 minutes
- Supports both HTTP requests and event-driven (via Eventarc)
- Supports WebSockets, HTTP/2, and gRPC
- Up to 32 GiB memory and 8 vCPUs per instance

When to Choose Cloud Run

- **Stateless web applications** and APIs
- **Microservices** that don't need Kubernetes complexity
- **Event-driven processing** (Pub/Sub, Cloud Storage events)

- **Background jobs** and data processing
- Applications with **variable traffic** (benefit from scale-to-zero)
- Teams that want **container flexibility without managing infrastructure**

When NOT to Choose Cloud Run

- Stateful applications requiring persistent local storage
- Applications needing GPUs
- Workloads requiring inter-container communication within a cluster
- Applications exceeding 60-minute request timeout

Cloud Run vs. Cloud Functions

Feature	Cloud Run	Cloud Functions
Unit of deployment	Container image	Function (source code)
Language support	Any (container-based)	Node.js, Python, Go, Java, .NET, Ruby, PHP
Request timeout	60 min	9 min (gen1) / 60 min (gen2)
Concurrent requests	Multiple per instance	1 per instance (gen1) / multiple (gen2)
Startup	Container startup	Cold start + function init
Customization	Full Dockerfile control	Limited to supported runtimes

6. Cloud Functions

What Are Cloud Functions?

- **Function as a Service (FaaS)** — write single-purpose functions
- Fully managed, serverless execution environment
- Automatically scales based on incoming events
- Pay only per invocation and compute time

Cloud Functions Generations

Feature	Gen 1	Gen 2 (recommended)
Max timeout	9 minutes	60 minutes
Max instances	1,000	1,000
Concurrency	1 request per instance	Up to 1,000 per instance
Traffic splitting	No	Yes
Min instances	Yes	Yes
Built on	Google's infrastructure	Cloud Run
Event sources	HTTP, Pub/Sub, Cloud Storage, Firestore, etc.	Same + Eventarc (100+ event sources)

When to Choose Cloud Functions

- **Simple event-driven processing** — React to cloud events
- **Webhooks** — Process incoming HTTP requests
- **Lightweight APIs** — Simple REST endpoints
- **Data transformation** — Process files uploaded to Cloud Storage
- **Scheduled tasks** — Combine with Cloud Scheduler
- **IoT data processing** — Process messages from Pub/Sub
- **Chatbots and notifications** — Lightweight processing

When NOT to Choose Cloud Functions

- Complex applications with multiple routes/handlers
- Applications requiring custom system libraries
- Workloads needing more than 32 GB memory
- Long-running processes (beyond 60 minutes for gen2)

7. Managed Instance Groups (MIGs) — Planning

MIGs are the standard pattern for scalable, self-healing stateless services on Compute Engine.

- Require an **instance template** (immutable); changing config means creating a new template and doing a rolling update
- **Stateless MIG**: no per-instance state; all instances interchangeable
- **Stateful MIG**: preserves per-instance disks, metadata, and IPs across recreation (for databases, stateful apps)
- **Regional MIG (multi-zone)**: distributes across zones in a region; recommended for production HA
  - Exam tip: Regional MIG + regional external HTTP(S) LB = highly available scalable architecture

Autoscaling Triggers

Signal	Use Case
CPU utilization	General compute workloads
HTTP load balancing utilization	When behind a load balancer
Cloud Monitoring metric (custom)	Queue depth, latency, etc.
Schedule-based	Predictable traffic spikes
Cloud Pub/Sub queue depth	Message processing workers

8. Preemptible and Spot VMs — Planning

- **Spot VMs**: up to 91% discount; can be reclaimed by GCP at any time with 30-second notice; no maximum runtime
- **Preemptible VMs**: legacy; same as Spot but with max 24-hour runtime cap
- Use only for: batch processing, fault-tolerant workloads, stateless/checkpointable jobs
- NOT suitable for: databases, stateful applications, latency-sensitive production services
- Planning decision: if workload can retry or checkpoint → use Spot; if it requires persistent uptime → use standard VM

9. GPU and TPU Planning

GPU Types and Use Cases

GPU	Use Case
NVIDIA T4	Inference, ML training, video transcoding
NVIDIA V100	ML training, HPC
NVIDIA A100	Large model training, highest performance
NVIDIA L4	Inference, graphics rendering

TPUs

- **Tensor Processing Units**: specialized for large-scale ML training/inference (especially TensorFlow)
- Available as TPU VMs (direct access) or via Vertex AI
- Planning tip: Use GPUs for general ML; use TPUs for very large models trained with TensorFlow/JAX

Availability

GPU availability varies by zone — check zone availability before architecture planning:

```
gcloud compute accelerator-types list --filter="zone:us-central1-a"
```

10. Compute Decision Flowchart (Summary)

```
Start
  |
  ├── Need full OS control or specific hardware (GPU, Windows)?
  |     └── YES → Compute Engine
  |
  ├── Already using Kubernetes or need container orchestration?
  |     └── YES → GKE
  |     └── GKE Standard (full control) or Autopilot (managed)?
  |
  ├── Containerized app, no K8s needed, stateless?
  |     └── YES → Cloud Run
  |
  ├── Simple function triggered by events?
  |     └── YES → Cloud Functions
  |
  └── Not sure?
        └── Cloud Run (most flexible serverless option)
```

11. Key Pricing Concepts

Compute Engine Pricing

- **On-demand** — Pay per second (minimum 1 minute)
- **Spot/Preemptible** — 60-91% discount, can be reclaimed
- **Committed Use Discounts** — 1-year (up to 57%) or 3-year (up to 70%) commitment
- **Sustained Use Discounts** — Automatic discount for running 25%+ of the month (up to 30%)
- **Free tier** — 1 e2-micro instance per month (us-west1, us-central1, us-east1)

GKE Pricing

- **Cluster management fee** — $0.10/hour for Standard (Autopilot: no cluster fee, pay per pod)
- **Node costs** — Standard Compute Engine pricing for worker node VMs
- **GKE Enterprise** — Additional per-vCPU fee

Cloud Run Pricing

- **Per request** + **CPU time** + **Memory time**
- Scale to zero = zero cost when idle
- Free tier: 2 million requests/month, 360,000 GiB-seconds memory, 180,000 vCPU-seconds

Cloud Functions Pricing

- **Per invocation** + **Compute time** (GB-seconds) + **Networking**
- Free tier: 2 million invocations/month, 400,000 GB-seconds, 200,000 GHz-seconds

---

Exam Practice Questions

1. **A startup needs to deploy a web application that receives unpredictable traffic, sometimes zero. They want to minimize costs and operational overhead. Which compute option is best?**
   - Answer: **Cloud Run** — Scales to zero (no cost when idle), automatic scaling for traffic spikes, minimal ops overhead, pay per request.
2. **An enterprise is migrating a legacy Windows application to GCP. The application requires specific OS configurations and cannot be containerized. Which compute option should they use?**
   - Answer: **Compute Engine** — Full OS control, supports Windows, can configure custom environments.
3. **A team wants to process images uploaded to Cloud Storage. Each image takes about 30 seconds. Which compute option is most cost-effective?**
   - Answer: **Cloud Functions (Gen 2)** — Event-driven (triggered by Cloud Storage upload), serverless, pay per invocation, quick execution time fits well.
4. **A company runs batch data processing jobs that take 4 hours each and can be restarted if interrupted. How can they reduce compute costs?**
   - Answer: Use **Compute Engine with Spot VMs** — 60-91% discount, acceptable for fault-tolerant batch jobs with checkpointing.
5. **Your application needs exactly 6 vCPUs and 24 GB of RAM. The closest predefined machine type has 8 vCPUs and 32 GB. What should you do?**
   - Answer: Use a **custom machine type** (`custom-6-24576`) to match exact requirements and avoid paying for unused resources.
6. **A team of 3 developers wants to run a microservices application with 12 services, auto-healing, rolling updates, and service discovery. Which compute option is best?**
   - Answer: **GKE (Autopilot mode)** — Built for microservices, provides all needed orchestration features, and Autopilot reduces operational burden.

Glossary

Anthos — Google's hybrid and multi-cloud application platform that enables GKE workloads to run on-premises or across multiple cloud providers; GKE Enterprise is the successor.

Autoscaler — GCP component that automatically adjusts the number of VM instances in a Managed Instance Group (MIG) or the number of nodes/pods in GKE based on defined metrics and policies.

Autopilot — A GKE cluster mode in which Google fully manages node provisioning, scaling, and security hardening; billing is per pod (CPU, memory, storage) rather than per node.

Batch Processing — Workload pattern where jobs process large volumes of data in discrete runs without real-time requirements; well-suited for Spot VMs due to fault tolerance.

BYOL (Bring Your Own License) — Licensing model where a customer uses pre-existing software licenses on GCP infrastructure, commonly associated with Sole-Tenant Nodes on Compute Engine.

CaaS (Container as a Service) — Cloud service model where the provider manages the container orchestration platform; GKE is GCP's CaaS offering.

Checkpointing — Technique of periodically saving a job's progress so it can resume from the last saved state if interrupted, essential for Spot VM workloads.

CI/CD (Continuous Integration / Continuous Delivery) — Software development practices that automate building, testing, and deploying code; CI/CD pipelines are a common Spot VM use case.

Cloud Functions — GCP's serverless Function as a Service (FaaS) offering; developers deploy individual functions that execute in response to HTTP triggers or cloud events without managing infrastructure.

Cloud Run — GCP's fully managed serverless platform for running containers; automatically scales from zero to thousands of instances and supports HTTP, WebSocket, HTTP/2, and gRPC.

Cloud Scheduler — GCP's fully managed cron job service for scheduling recurring tasks such as invoking Cloud Functions or HTTP endpoints on a time-based schedule.

Cluster — In GKE, a collection of nodes managed by a Kubernetes control plane; the top-level organizational unit for containerized workloads.

Committed Use Discounts (CUDs) — Discounts of up to 57% (1-year) or 70% (3-year) on Compute Engine CPU and memory in exchange for a committed usage contract; purchased at the project level.

Compute Engine — GCP's Infrastructure as a Service (IaaS) offering providing customizable virtual machines running on Google's global infrastructure; supports GPUs, TPUs, and Sole-Tenant Nodes.

Compute-Optimized Machine Family — Compute Engine VM family (C2, C2D, H3) designed for CPU-intensive workloads such as HPC, gaming servers, batch processing, and media transcoding.

Container — Lightweight, portable unit that packages application code with its dependencies; the fundamental deployment unit for GKE and Cloud Run.

Control Plane — The Kubernetes management layer responsible for scheduling pods, maintaining cluster state, and serving the Kubernetes API; Google manages this in GKE.

Custom Machine Type — A Compute Engine VM configuration with a user-specified (non-standard) number of vCPUs and amount of memory, available for N1, N2, N2D, and E2 families.

Dataproc — GCP's managed Apache Hadoop and Spark service; commonly used with Spot VMs for cost-effective big data processing.

Deployment — Kubernetes workload object that declaratively manages a set of identical pod replicas, supporting rolling updates and rollbacks.

E2 — Compute Engine general-purpose machine family offering cost-optimized VMs suitable for balanced workloads such as web servers, development environments, and small databases.

Eventarc — GCP service that routes events from over 100 GCP services and custom sources to Cloud Run, Cloud Functions Gen 2, and other targets, enabling event-driven architectures.

FaaS (Function as a Service) — Serverless compute model in which individual functions are deployed and executed in response to events; Cloud Functions is GCP's FaaS offering.

Free Tier — A monthly allowance of GCP resources at no cost, including one e2-micro Compute Engine instance in select US regions, 2 million Cloud Run requests, and 2 million Cloud Functions invocations.

General-Purpose Machine Family — Compute Engine VM family (E2, N1, N2, N2D, T2D, T2A, C3, C3D) offering a balanced ratio of CPU and memory for standard workloads.

GKE (Google Kubernetes Engine) — Google Cloud's managed Kubernetes service for deploying, scaling, and operating containerized applications; available in Standard and Autopilot modes.

GKE Enterprise — Enhanced tier of GKE providing additional features for multi-cluster management, policy enforcement, and hybrid/multi-cloud deployments, formerly part of Anthos.

GPU (Graphics Processing Unit) — Specialized hardware accelerator used for machine learning training and inference, video transcoding, and HPC; available on Accelerator-Optimized Compute Engine VMs.

gRPC — High-performance open-source RPC framework using Protocol Buffers; supported by Cloud Run for inter-service communication.

Hadoop — Open-source framework for distributed storage and processing of large datasets; commonly run on Dataproc or Spot VMs.

HPC (High-Performance Computing) — Workloads requiring significant computational power, often using Compute-Optimized or Accelerator-Optimized VM families with low-latency networking.

HTTP/2 — Binary HTTP protocol offering multiplexed streams and header compression; supported natively by Cloud Run.

IaaS (Infrastructure as a Service) — Cloud service model providing virtualized computing resources such as VMs, storage, and networking; Compute Engine is GCP's IaaS offering.

Instance Template — An immutable resource that defines the configuration (machine type, disk, image, network, metadata) for VMs in a Managed Instance Group; changing config requires creating a new template.

JAX — Google's open-source numerical computation library used for machine learning research; optimized for TPU execution.

Knative — Open-source Kubernetes-based platform for serverless workloads; Cloud Run is built on Knative.

Kubernetes — Open-source container orchestration system for automating deployment, scaling, and management of containerized applications; the foundation of GKE.

kubectl — Command-line tool for interacting with Kubernetes clusters; used to deploy applications, inspect resources, and manage GKE workloads.

Lift-and-Shift — Cloud migration strategy that moves on-premises workloads to cloud VMs (Compute Engine) with minimal changes to the application architecture.

Local SSD — Physically attached NVMe or SCSI SSD on a Compute Engine host; offers extremely high IOPS but is ephemeral (data lost on VM stop or delete).

M1 / M2 / M3 — Compute Engine memory-optimized machine families designed for large in-memory workloads such as SAP HANA and in-memory databases.

Managed Instance Group (MIG) — A group of identical Compute Engine VM instances managed as a single entity, supporting autoscaling, auto-healing, rolling updates, and multi-zone distribution.

Memory-Optimized Machine Family — Compute Engine VM family (M1, M2, M3) providing very high memory-to-CPU ratios for SAP HANA, in-memory databases, and large analytics workloads.

Metadata Server — GCP service accessible from within a VM at `metadata.google.internal`; used to retrieve instance metadata including preemption notices for Spot VMs.

Microservices — Architectural style that structures an application as a collection of loosely coupled, independently deployable services; GKE and Cloud Run are common deployment targets.

N1 / N2 / N2D — Compute Engine general-purpose machine families; N1 supports the widest range of accelerators; N2/N2D offer higher performance per core.

NVIDIA A100 — High-performance GPU available on Compute Engine A3 instances; used for large-scale ML model training.

NVIDIA L4 — GPU available on Compute Engine G2 instances; optimized for inference workloads and graphics rendering.

NVIDIA T4 — GPU available on Compute Engine; used for ML inference, training, and video transcoding.

NVIDIA V100 — GPU available on Compute Engine; used for ML training and HPC workloads.

Node — In GKE, a virtual machine that runs pods; nodes are grouped into node pools and form the compute capacity of a cluster.

Node Pool — A group of nodes within a GKE cluster that share the same configuration (machine type, OS image, disk type); clusters can have multiple node pools.

On-Demand Pricing — Standard Compute Engine billing at per-second rates (minimum 1 minute) with no upfront commitment or usage requirement.

PaaS (Platform as a Service) — Cloud service model where the provider manages the underlying infrastructure and runtime; App Engine is GCP's primary PaaS offering.

Pod — The smallest deployable unit in Kubernetes, consisting of one or more containers that share network and storage resources.

Preemptible VM — Legacy low-cost Compute Engine VM type offering a fixed discount; Google can reclaim it at any time with a 30-second warning and it has a maximum runtime of 24 hours; superseded by Spot VMs.

Pub/Sub (Cloud Pub/Sub) — GCP's fully managed asynchronous messaging service; used as an autoscaling trigger signal (queue depth) for MIGs and as an event source for Cloud Functions and Cloud Run.

Regional MIG — A Managed Instance Group distributed across multiple zones within a single region; provides higher availability than a zonal MIG.

SAP HANA — In-memory enterprise database and application platform requiring very high RAM; suited for Memory-Optimized Compute Engine VM families.

Service (Kubernetes) — A Kubernetes resource that provides a stable network endpoint (IP address and DNS name) for accessing a dynamic set of pods.

Serverless — Computing model in which the cloud provider automatically manages infrastructure provisioning and scaling; Cloud Run and Cloud Functions are GCP's primary serverless offerings.

Shutdown Script — A script configured on a Compute Engine VM that executes when the VM receives a termination signal (including Spot VM preemption), allowing graceful cleanup.

Sole-Tenant Node — A physical Compute Engine server dedicated exclusively to a single customer's VMs; used for compliance, licensing (BYOL), and performance isolation requirements.

Spark — Open-source distributed data processing engine; commonly run on Dataproc or Spot VMs for large-scale data transformations.

Spot VM — Low-cost Compute Engine VM offering up to 91% discount; Google can reclaim it at any time with a 30-second notice; has no maximum runtime limit (unlike Preemptible VMs).

Stateful MIG — A Managed Instance Group that preserves per-instance state (disks, metadata, IP addresses) across VM recreation; suitable for databases and stateful applications.

Stateless MIG — A Managed Instance Group where all instances are identical and interchangeable; no per-instance state is preserved; the standard pattern for scalable web services.

StatefulSet — Kubernetes workload object for managing stateful applications, providing stable network identities and persistent storage per pod across rescheduling.

Storage-Optimized Machine Family — Compute Engine VM family (Z3) designed for high IOPS workloads requiring large local SSD capacity.

Sustained Use Discounts (SUDs) — Automatic Compute Engine discounts of up to 30% applied when a VM runs for more than 25% of a billing month; requires no commitment.

T2A / T2D — Compute Engine general-purpose machine families; T2A uses Arm-based Ampere Altra processors; T2D uses AMD EPYC processors.

TensorFlow — Google's open-source machine learning framework; the primary framework optimized for TPU execution.

TPU (Tensor Processing Unit) — Google-designed ASIC optimized for large-scale machine learning workloads, particularly TensorFlow and JAX; available as TPU VMs or via Vertex AI.

vCPU (Virtual CPU) — A virtualized processing core assigned to a Compute Engine VM; the primary unit for specifying VM compute capacity and for pricing.

Vertex AI — GCP's unified machine learning platform providing managed services for ML model training and serving, including TPU access.

VM (Virtual Machine) — A software emulation of a physical computer running an operating system and applications; the fundamental unit of Compute Engine.

WebSocket — Protocol providing full-duplex communication over a single TCP connection; supported by Cloud Run for real-time applications.

Windows Server — Microsoft's server operating system; supported by Compute Engine, making it a valid choice for Windows-based workloads requiring full OS control.

Zonal MIG — A Managed Instance Group where all VM instances reside within a single zone; simpler but less resilient than a Regional MIG.